

Apuntes de Laravel

Índice

1. [Gestión de Sesiones](#)
 2. [Gestión de Usuarios con Breeze](#)
 3. [Gestión de Errores](#)
-

Gestión de Sesiones

Introducción

Laravel proporciona un sistema de sesiones unificado que permite almacenar información del usuario entre peticiones HTTP. Las sesiones se pueden almacenar en diferentes drivers: archivos, bases de datos, cookies, Redis, Memcached, etc.

Configuración

La configuración de sesiones se encuentra en `config/session.php` :

```
'driver' => env('SESSION_DRIVER', 'file'),
'lifetime' => 120, // minutos
'expire_on_close' => false,
'encrypt' => false,
'files' => storage_path('framework/sessions'),
'connection' => null,
'table' => 'sessions',
'store' => null,
'lottery' => [2, 100],
'cookie' => env('SESSION_COOKIE', 'laravel_session'),
'path' => '/',
'domain' => env('SESSION_DOMAIN', null),
'secure' => env('SESSION_SECURE_COOKIE', false),
'http_only' => true,
'same_site' => 'lax',
```

Almacenar Datos en Sesión

Usando el helper `session()` :

```
// Almacenar un valor
session(['key' => 'value']);

// Almacenar múltiples valores
session([
    'nombre' => 'Juan',
    'edad' => 25
]);
```

Usando la fachada `Session` :

```
use Illuminate\Support\Facades\Session;

Session::put('key', 'value');
Session::put('usuario', ['nombre' => 'Ana', 'rol' => 'admin']);
```

Usando el objeto Request:

```
public function store(Request $request)
{
    $request->session()->put('key', 'value');
}
```

Recuperar Datos de Sesión

```
// Usando el helper
$value = session('key');
$value = session('key', 'default'); // Con valor por defecto

// Usando la fachada
$value = Session::get('key');
$value = Session::get('key', 'default');

// Usando Request
$value = $request->session()->get('key');

// Recuperar todos los datos
$data = session()->all();
```

Verificar Existencia

```
if (session()->has('key')) {
    // La clave existe
}

if (session()->exists('key')) {
    // Existe incluso si es null
}

if (session()->missing('key')) {
    // La clave no existe
}
```

Datos Flash

Los datos flash se almacenan solo para la siguiente petición:

```
// Almacenar dato flash
session()->flash('mensaje', 'Operación exitosa');
```

```
$request->session()->flash('status', 'Tarea completada');

// Mantener datos flash por una petición más
session()->reflash();

// Mantener solo algunos datos
session()->keep(['username', 'email']);
```

Eliminar Datos

```
// Eliminar una clave específica
session()->forget('key');
session()->forget(['key1', 'key2']);

// Obtener y eliminar
$value = session()->pull('key');

// Limpiar toda la sesión
session()->flush();
```

Regenerar ID de Sesión

Importante para prevenir ataques de fijación de sesión:

```
$request->session()->regenerate();

// Invalidar sesión actual y generar nueva
$request->session()->invalidate();
```

Driver de Base de Datos

Para usar sesiones en base de datos:

1. Crear la tabla de sesiones:

```
php artisan session:table
php artisan migrate
```

2. Configurar en `.env` :

```
SESSION_DRIVER=database
```

Ejemplo Práctico: Carrito de Compras

```
class CarritoController extends Controller
{
    public function agregar(Request $request, $productoId)
    {
```

```

        $carrito = session()->get('carrito', []);

        if (isset($carrito[$productoId])) {
            $carrito[$productoId]['cantidad']++;
        } else {
            $carrito[$productoId] = [
                'nombre' => $request->nombre,
                'precio' => $request->precio,
                'cantidad' => 1
            ];
        }

        session()->put('carrito', $carrito);
        session()->flash('success', 'Producto agregado al carrito');

        return redirect()->back();
    }

    public function ver()
    {
        $carrito = session()->get('carrito', []);
        return view('carrito.index', compact('carrito'));
    }

    public function vaciar()
    {
        session()->forget('carrito');
        return redirect()->route('carrito.ver');
    }
}

```

Gestión de Usuarios con Breeze

¿Qué es Laravel Breeze?

Laravel Breeze es un starter kit minimalista que implementa autenticación completa: registro, login, recuperación de contraseña, verificación de email y gestión de perfil.

Instalación

```

# Instalar Breeze
composer require laravel/breeze --dev

# Publicar archivos (elegir stack)
php artisan breeze:install

# Opciones disponibles:
# - blade (tradicional con Blade)
# - react (con Inertia.js y React)
# - vue (con Inertia.js y Vue)

```

```
# - api (solo API sin frontend)

# Instalar dependencias y compilar assets
npm install
npm run dev

# Ejecutar migraciones
php artisan migrate
```

Estructura de Archivos Generados

```
app/
├── Http/
│   └── Controllers/
│       └── Auth/
│           ├── AuthenticatedSessionController.php
│           ├── ConfirmablePasswordController.php
│           ├── EmailVerificationNotificationController.php
│           ├── EmailVerificationPromptController.php
│           ├── NewPasswordController.php
│           ├── PasswordController.php
│           ├── PasswordResetLinkController.php
│           ├── RegisteredUserController.php
│           └── VerifyEmailController.php
└── resources/
    ├── views/
    │   └── auth/
    │       ├── confirm-password.blade.php
    │       ├── forgot-password.blade.php
    │       ├── login.blade.php
    │       ├── register.blade.php
    │       ├── reset-password.blade.php
    │       └── verify-email.blade.php
    └── routes/
        ├── auth.php
        └── web.php
```

Rutas de Autenticación

En `routes/auth.php` se definen todas las rutas:

```
// Registro
Route::get('/register', [RegisteredUserController::class, 'create']);
Route::post('/register', [RegisteredUserController::class, 'store']);

// Login
Route::get('/login', [AuthenticatedSessionController::class, 'create'])
    ->name('login');
Route::post('/login', [AuthenticatedSessionController::class, 'store']);
```

```
// Logout
Route::post('/logout', [AuthenticatedSessionController::class, 'destroy'])
    ->name('logout');

// Recuperar contraseña
Route::get('/forgot-password', [PasswordResetLinkController::class, 'create']);
Route::post('/forgot-password', [PasswordResetLinkController::class, 'store']);

// Reset contraseña
Route::get('/reset-password/{token}', [NewPasswordController::class, 'create']);
Route::post('/reset-password', [NewPasswordController::class, 'store']);
```

Middleware de Autenticación

Proteger rutas:

```
// En routes/web.php
Route::middleware('auth')->group(function () {
    Route::get('/dashboard', function () {
        return view('dashboard');
    })->name('dashboard');

    Route::get('/profile', [ProfileController::class, 'edit'])->
    name('profile.edit');
});
```

Verificación de email:

```
Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('/settings', [SettingsController::class, 'index']);
});
```

Registro de Usuarios

Controlador de Registro (`RegisteredUserController`):

```
public function store(Request $request): RedirectResponse
{
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
    ]);
```

```

        event(new Registered($user));

        Auth::login($user);

        return redirect(RouteServiceProvider::HOME);
    }

```

Login de Usuarios

Controlador de Sesión (`AuthenticatedSessionController`):

```

public function store(LoginRequest $request): RedirectResponse
{
    $request->authenticate();

    $request->session()->regenerate();

    return redirect()->intended(RouteServiceProvider::HOME);
}

```

Request personalizado:

```

class LoginRequest extends FormRequest
{
    public function authenticate(): void
    {
        $this->ensureIsNotRateLimited();

        if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
            RateLimiter::hit($this->throttleKey());

            throw ValidationException::withMessages([
                'email' => __('auth.failed'),
            ]);
        }

        RateLimiter::clear($this->throttleKey());
    }
}

```

Verificación de Email

Habilitar verificación en el modelo User:

```

use Illuminate\Contracts\Auth\MustVerifyEmail;

class User extends Authenticatable implements MustVerifyEmail
{

```

```
// ...  
}
```

Proteger rutas:

```
Route::middleware(['auth', 'verified'])->group(function () {  
    // Rutas que requieren email verificado  
});
```

Enviar notificación de verificación:

```
$user->sendEmailVerificationNotification();
```

Recuperación de Contraseña

Proceso completo:

1. Usuario solicita reset en `/forgot-password`
2. Laravel envía email con token
3. Usuario hace clic en enlace con token
4. Usuario ingresa nueva contraseña en `/reset-password/{token}`
5. Contraseña se actualiza

Personalizar email:

```
// En app/Providers/AuthServiceProvider.php  
use Illuminate\Auth\Notifications\ResetPassword;  
  
public function boot()  
{  
    ResetPassword::createUrlUsing(function ($user, string $token) {  
        return 'https://example.com/reset-password?token='.$token;  
    });  
}
```

Gestión de Perfil

Actualizar información:

```
public function update(Request $request)  
{  
    $request->validate([  
        'name' => ['required', 'string', 'max:255'],  
        'email' => ['required', 'email', 'max:255', 'unique:users,email, '.$request->user()->id],  
    ]);  
  
    $request->user()->fill($request->only('name', 'email'));  
  
    if ($request->user()->isDirty('email')) {
```



```

        $request->user()->email_verified_at = null;
    }

    $request->user()->save();

    return redirect()->route('profile.edit')->with('status', 'profile-updated');
}

```

Actualizar contraseña:

```

public function update(Request $request)
{
    $validated = $request->validate([
        'current_password' => ['required', 'current_password'],
        'password' => ['required', Rules\Password::defaults(), 'confirmed'],
    ]);

    $request->user()->update([
        'password' => Hash::make($validated['password']),
    ]);

    return back()->with('status', 'password-updated');
}

```

Eliminar cuenta:

```

public function destroy(Request $request)
{
    $request->validate([
        'password' => ['required', 'current_password'],
    ]);

    $user = $request->user();

    Auth::logout();

    $user->delete();

    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/');
}

```

Autenticación en Blade

Verificar autenticación:

```

@auth
    <p>Bienvenido, {{ Auth::user()->name }}</p>

```

```
@endauth
```

```
@guest
```

```
<a href="{{ route('login') }}">Iniciar sesión</a>
```

```
<a href="{{ route('register') }}">Registrarse</a>
```

```
@endguest
```

Acceder al usuario:

```
<p>Email: {{ auth()->user()->email }}</p>
```

```
<p>Nombre: {{ Auth::user()->name }}</p>
```

Personalizar Breeze

Agregar campos al registro:

1. Agregar campo en la migración:

```
$table->string('telefono')->nullable();
```

2. Modificar formulario de registro:

```
<div>
    <x-input-label for="telefono" :value="__('Teléfono')" />
    <x-text-input id="telefono" name="telefono" type="text" class="mt-1 block w-
full" />
    <x-input-error :messages="$errors->get('telefono')" class="mt-2" />
</div>
```

3. Actualizar validación y creación:

```
$request->validate([
    'name' => ['required', 'string', 'max:255'],
    'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
    'telefono' => ['nullable', 'string', 'max:20'],
    'password' => ['required', 'confirmed', Rules\Password::defaults()],
]);

$user = User::create([
    'name' => $request->name,
    'email' => $request->email,
    'telefono' => $request->telefono,
    'password' => Hash::make($request->password),
]);
```

4. Agregar al fillable del modelo:

```
protected $fillable = [
    'name',
```

```
'email',  
'telefono',  
'password',  
];
```

Roles y Permisos (Extensión)

Aunque Breeze no incluye roles, puedes agregarlos:

```
// Migración  
Schema::create('roles', function (Blueprint $table) {  
    $table->id();  
    $table->string('name')->unique();  
    $table->timestamps();  
});  
  
Schema::table('users', function (Blueprint $table) {  
    $table->foreignId('role_id')->nullable()->constrained();  
});  
  
// Modelo User  
public function role()  
{  
    return $this->belongsTo(Role::class);  
}  
  
public function hasRole($role)  
{  
    return $this->role && $this->role->name === $role;  
}  
  
// Middleware  
public function handle($request, Closure $next, $role)  
{  
    if (!auth()->check() || !auth()->user()->hasRole($role)) {  
        abort(403);  
    }  
    return $next($request);  
}  
  
// Uso en rutas  
Route::middleware(['auth', 'role:admin'])->group(function () {  
    Route::get('/admin/dashboard', [AdminController::class, 'index']);  
});
```

Gestión de Errores

Introducción

Laravel proporciona un sistema robusto de manejo de excepciones y errores, centralizado en el handler de excepciones.

Configuración

La configuración se encuentra en `config/app.php` :

```
'debug' => env('APP_DEBUG', false),
```

⚠ **IMPORTANTE:** En producción, `APP_DEBUG` debe estar en `false` .

Handler de Excepciones

El archivo principal es `app/Exceptions/Handler.php` :

```
namespace App\Exceptions;

use Illuminate\Foundation\Exceptions\Handler as ExceptionHandler;
use Throwable;

class Handler extends ExceptionHandler
{
    protected $dontReport = [
        // Excepciones que no se reportan
    ];

    protected $dontFlash = [
        'current_password',
        'password',
        'password_confirmation',
    ];

    public function register()
    {
        $this->reportable(function (Throwable $e) {
            // Lógica de reporte personalizada
        });
    }
}
```

Reportar Excepciones

Método `report()` :

```
public function register()
{
    $this->reportable(function (InvalidOrderException $e) {
        Log::error('Pedido inválido: ' . $e->getMessage(), [
            'order_id' => $e->order->id,
            'user_id' => $e->user->id,
        ]);
    });
}
```

```
});  
}
```

No reportar ciertas excepciones:

```
protected $dontReport = [  
    \Illuminate\Auth\AuthenticateException::class,  
    \Illuminate\Validation\ValidationException::class,  
];
```

Reportar a servicios externos:

```
public function register()  
{  
    $this->reportable(function (Throwable $e) {  
        if (app()->bound('sentry')) {  
            app('sentry')->captureException($e);  
        }  
    });  
}
```

Renderizar Excepciones

Personalizar respuesta de errores:

```
public function register()  
{  
    $this->renderable(function (NotFoundException $e, $request) {  
        if ($request->is('api/*')) {  
            return response()->json([  
                'message' => 'Recurso no encontrado'  
            ], 404);  
        }  
    });  
  
    $this->renderable(function (CustomException $e, $request) {  
        return response()->view('errors.custom', [  
            'message' => $e->getMessage()  
        ], 500);  
    });  
}
```

Páginas de Error Personalizadas

Laravel busca vistas en `resources/views/errors/` según el código HTTP:

```
resources/views/errors/  
├─ 404.blade.php  
├─ 403.blade.php
```

```
|— 500.blade.php
|— 503.blade.php
```

Ejemplo de 404.blade.php:

```
@extends('layouts.app')

@section('content')
<div class="error-page">
    <h1>404</h1>
    <h2>Página no encontrada</h2>
    <p>Lo sentimos, la página que buscas no existe.</p>
    <a href="{{ route('home') }}" class="btn">Volver al inicio</a>
</div>
@endsection
```

Vista genérica para cualquier error:

```
{{-- resources/views/errors/layout.blade.php --}}
<!DOCTYPE html>
<html>
<head>
    <title>@yield('title')</title>
</head>
<body>
    <div class="container">
        <div class="content">
            <div class="title">@yield('code')</div>
            <div class="message">@yield('message')</div>
        </div>
    </div>
</body>
</html>
```

Logging

Laravel usa Monolog para logging. Configuración en `config/logging.php` :

```
'channels' => [
    'stack' => [
        'driver' => 'stack',
        'channels' => ['single', 'slack'],
    ],

    'single' => [
        'driver' => 'single',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
    ],

    'daily' => [
```

```

        'driver' => 'daily',
        'path' => storage_path('logs/laravel.log'),
        'level' => env('LOG_LEVEL', 'debug'),
        'days' => 14,
    ],
],

```

Niveles de log:

```

use Illuminate\Support\Facades\Log;

Log::emergency($message); // Sistema inutilizable
Log::alert($message);     // Acción inmediata requerida
Log::critical($message);  // Condiciones críticas
Log::error($message);     // Errores de ejecución
Log::warning($message);   // Advertencias
Log::notice($message);    // Normal pero significativo
Log::info($message);      // Información
Log::debug($message);     // Información de depuración

```

Uso con contexto:

```

Log::info('Usuario registrado', [
    'user_id' => $user->id,
    'email' => $user->email
]);

Log::error('Error al procesar pago', [
    'exception' => $e->getMessage(),
    'trace' => $e->getTraceAsString(),
    'user_id' => auth()->id()
]);

```

Canales específicos:

```

Log::channel('slack')->critical('Error crítico en producción');
Log::stack(['single', 'slack'])->info('Información importante');

```

Excepciones Personalizadas

Crear excepción:

```
php artisan make:exception CustomException
```

Ejemplo de excepción personalizada:

```

namespace App\Exceptions;

use Exception;

```

```

class InsufficientFundsException extends Exception
{
    protected $account;
    protected $amount;

    public function __construct($account, $amount)
    {
        $this->account = $account;
        $this->amount = $amount;

        parent::__construct("Fondos insuficientes en la cuenta {$account->id}");
    }

    public function report()
    {
        Log::warning('Intento de pago con fondos insuficientes', [
            'account_id' => $this->account->id,
            'balance' => $this->account->balance,
            'amount_requested' => $this->amount,
        ]);
    }

    public function render($request)
    {
        return response()->view('errors.insufficient-funds', [
            'balance' => $this->account->balance,
            'amount' => $this->amount,
        ], 422);
    }
}

```

Lanzar la excepción:

```

if ($account->balance < $amount) {
    throw new InsufficientFundsException($account, $amount);
}

```

Validación y Errores

Errores de validación:

```

$request->validate([
    'email' => 'required|email',
    'password' => 'required|min:8',
]);

```

Manejo manual:


```

use Illuminate\Support\Facades\Validator;

$validator = Validator::make($request->all(), [
    'email' => 'required|email',
    'password' => 'required|min:8',
]);

if ($validator->fails()) {
    return redirect('register')
        ->withErrors($validator)
        ->withInput();
}

```

Mensajes de error personalizados:

```

$messages = [
    'email.required' => 'El correo electrónico es obligatorio',
    'email.email' => 'Debe ser un correo válido',
    'password.min' => 'La contraseña debe tener al menos :min caracteres',
];

$validator = Validator::make($request->all(), $rules, $messages);

```

Mostrar errores en Blade:

```

@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

{{-- Error específico --}}
@error('email')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror

{{-- Con componentes Blade --}}
<x-input-error :messages="$errors->get('email')" class="mt-2" />

```

Errores HTTP

Lanzar errores HTTP manualmente:

```

abort(404);
abort(403, 'Acción no autorizada');
abort(500, 'Error interno del servidor');

```

```
abort_if($user->id !== $post->user_id, 403);
abort_unless(Auth::check(), 401);
```

Crear respuesta de error:

```
return response()->view('errors.custom', [], 500);
return response()->json(['error' => 'No encontrado'], 404);
```

Modo Mantenimiento

Activar:

```
php artisan down
php artisan down --message="Actualización en curso" --retry=60
php artisan down --secret="mi-token-secreto"
```

Desactivar:

```
php artisan up
```

Vista personalizada (503.blade.php):

```
@extends('layouts.app')

@section('content')
<div class="maintenance">
    <h1>Sitio en Mantenimiento</h1>
    <p>Volveremos pronto. Estamos mejorando nuestros servicios.</p>
</div>
@endsection
```

Debugging y Herramientas

dd() y dump():

```
dd($variable); // Die and dump
dump($variable); // Dump y continúa

// En Blade
@dd($users)
@dump($user)
```

Laravel Debugbar:

```
composer require barryvdh/laravel-debugbar --dev
```

Laravel Telescope (para desarrollo):

```
composer require laravel/telescope --dev
php artisan telescope:install
php artisan migrate
```

Try-Catch en Controladores

Ejemplo práctico:

```
public function store(Request $request)
{
    try {
        DB::beginTransaction();

        $user = User::create($request->validated());
        $user->profile()->create($request->profile);

        DB::commit();

        return redirect()->route('users.index')
            ->with('success', 'Usuario creado exitosamente');
    } catch (\Exception $e) {
        DB::rollBack();

        Log::error('Error al crear usuario', [
            'exception' => $e->getMessage(),
            'data' => $request->all()
        ]);

        return back()
            ->withInput()
            ->withErrors(['error' => 'Error al crear el usuario. Intente nuevamente.']);
    }
}
```

Best Practices

1. Nunca mostrar detalles de excepciones en producción
2. Usar códigos HTTP apropiados
3. Loggear errores importantes con contexto
4. Crear excepciones personalizadas para lógica de negocio
5. Usar try-catch solo cuando se pueda manejar el error
6. Personalizar páginas de error para mejor UX
7. Monitorear logs regularmente
8. Usar transacciones de BD en operaciones críticas

Ejemplo Completo de Manejo de Errores

```
// app/Exceptions/PaymentException.php
class PaymentException extends Exception
{
    public function render($request)
    {
        if ($request->expectsJson()) {
            return response()->json([
                'error' => 'Error en el pago',
                'message' => $this->getMessage()
            ], 422);
        }

        return redirect()->back()
            ->withInput()
            ->with('error', 'No se pudo procesar el pago: ' . $this->getMessage());
    }

    public function report()
    {
        Log::channel('payments')->error('Error de pago', [
            'message' => $this->getMessage(),
            'trace' => $this->getTraceAsString(),
        ]);
    }
}

// Uso en controlador
public function processPayment(Request $request)
{
    try {
        $payment = PaymentService::process($request->all());

        return response()->json([
            'success' => true,
            'payment_id' => $payment->id
        ]);
    } catch (InsufficientFundsException $e) {
        throw new PaymentException('Fondos insuficientes');
    } catch (InvalidCardException $e) {
        throw new PaymentException('Tarjeta inválida');
    } catch (\Exception $e) {
        Log::critical('Error inesperado en pago', [
            'exception' => $e
        ]);
        throw new PaymentException('Error al procesar el pago');
    }
}
```

Recursos Adicionales

- [Documentación Oficial de Laravel](#)
- [Laravel Breeze Documentation](#)
- [Laravel Session Documentation](#)
- [Laravel Error Handling](#)
- [Laracasts](#) - Tutoriales en video
- [Laravel News](#) - Noticias y tutoriales